PYTHON PROGRAMMING



21 Days Challenge

Day-06



Day-06

TUPLES AND SETS: KEY CONCEPTS, ACCESS, MANIPULATION, OPERATORS, AND HANDS-ON EXERCISES

Instructor



Abdullah Al Zubaer

Instructor – Artificial Intelligence & Data Science

B.Sc. in Internet of Things and Robotics Engineering

Gazipur Digital University, Gazipur, Bangladesh

Table Of Content

Tuples: Definition and Properties

Tuples: Access and Modification

Tuples: Operations and Functions

Tuple Unpacking

Sets: Definition and Properties

Sets: Access and Modification

Sets: Operations and Functions

Tuples

A tuple is a built-in data type in Python that is used to store multiple items in a single variable.

```
FRUITS = ('ORANGE','PEAR', 'BANANA','APPLE', 'BANANA')
```

- ORDERED
- UNCHANGEABLE (IMMUTABLE)
- ALLOWS DUPLICATES

"A SINGLE-ELEMENT TUPLE MUST HAVE A TRAILING COMMA (',') TO BE RECOGNIZED AS A TUPLE."

Access and Modification

- Items can be accessed using indexing, similar to lists.
- Tuples cannot be updated or modified after creation.

Operations and Functions

Concatenation:

```
tuple1 = (1, 2, 3)
tuple2 = (4, 5, 6)
result_concat = tuple1 + tuple2
print("Concatenation:", result_concat)
```

Output

```
Concatenation: (1, 2, 3, 4, 5, 6)
```

Operations and Functions

```
my_tuple = (5, 10, 3, 8, 2)
my_list = [1, 4, 7]
print("Tuple:", my_tuple)
print("Length:", len(my_tuple))
print("Maximum:", max(my_tuple))
print("Minimum:", min(my_tuple))
print("Converted Tuple from list:", tuple(my_list))
```

Output

```
Tuple: (5, 10, 3, 8, 2)
Length: 5
Maximum: 10
Minimum: 2
Converted Tuple from list: (1, 4, 7)
```

Tuples Unpacking

Assign tuple elements to individual variables directly.

```
tuple = (10, 20, 30)
x, y, z = tuple
print(f"x: {x}, y: {y}, z: {z}")
```

Output

```
x: 10, y: 20, z: 30
```

Sets

A set in Python is a built-in collection data type that stores unordered, unique elements

```
FRUITS = {'ORANGE','PEAR', 'BANANA','APPLE'}
```

- UNORDERED
- MUTABLE
- DUPLICATES NOT ALLOWS(UNIQUE)

Creating Set Items

```
set1 = {1, 2, 3, 2, 1}

set2 = set([4, 5, 5, 6])

print(set1)
print(set2)
```

Output

```
{1, 2, 3}
{4, 5, 6}
```

Accessing set items

for loop

```
fruits = {'orange','pear', 'banana','apple'}

for set in fruits:
    print(set)
```

Output

```
banana
orange
apple
pear
```

Accessing set items

List Comprehension

```
fruits = {'orange','pear', 'banana','apple'}

[print(fruit) for fruit in fruits]
```

Output

```
pear
banana
orange
apple
```

Adding set items

```
s = set()
s.add("Python")  # adds single elements

s = {1, 2}
s.update([3, 4]) # adds multiple elements
```

Removing set items

```
remove(item)
discard(item)
pop()
clear()
```

Set operation

```
set1 = {1, 2, 3}
set2 = {2, 3, 4}

print(set1.union(set2)) # Using union() method
print( set1 | set2) # Using | operator

print( set1.intersection(set2))# Using intersection() method
print( set1 & set2) # Using & operator


print( set1.difference(set2)) # Using difference() method
print( set1 - set2)# Using - operator
```

Output

```
{1, 2, 3, 4}
{1, 2, 3, 4}
{2, 3}
{2, 3}
{1}
{1}
```

Set operation

Output

```
{1, 2, 3, 4}
{1, 2, 3, 4}


{2, 3}
{2, 3}


{1}
{1}
```

QUESTION SESSION

THANK YOU

https://gobeshona.com.bd/